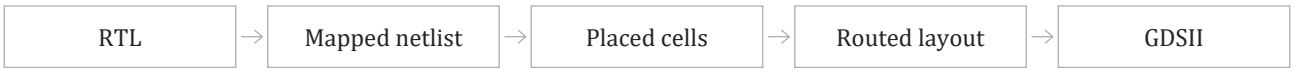
RTL to GDS Flow

Implementation sequence · Iteration and course approach

Implementation sequence, stage purpose and the NPTEL teaching approach.



1. Define intent and architecture

Set function, interfaces and power/performance/area targets; partition the design into blocks. Architecture determines what hardware must do. High-level synthesis is one route from behavioral operations to a datapath and controller; RTL can also be written directly. L03-L04

2. Model and verify RTL

Describe bit-accurate combinational logic and state transitions. Simulation exercises behavior with a testbench; formal methods check specified properties or equivalence under stated assumptions. Resolve intent before implementation. Output: verified synthesizable RTL and its requirements. L11; L13

3. Synthesize a technology-mapped netlist

Elaborate and infer hardware. Resolve hierarchy, parameters and widths; turn RTL constructs into registers, multiplexers, arithmetic and control networks. L15-L16

Optimize logic. Simplify Boolean covers, factor/share expressions and optimize state representations while preserving required behavior. Cost depends on both logic structure and its implementation. L17; L19-L20

Map to technology. Select legal cells from a characterized library under timing, load and area objectives. Output: a cell-instance netlist with connectivity. Constraints and library models guide optimization from this stage onward. L26; L32

4. Make the design testable

Plan DFT early; add appropriate scan/test structures to improve internal controllability and observability. Generate tests for defined fault models and recheck functional/test modes and timing. Scan insertion and physical scan-chain reordering belong to coordinated logical and physical stages. L39-L42

5. Implement the physical design

Floorplan and power plan: set die/core organization, macro locations, pin access and the supply network; provide space and electrical support for implementation.

Place and legalize: assign legal cell locations, balancing wirelength, congestion and timing. **Clock tree synthesis:** distribute clocks with controlled skew, latency and transition.

Route: allocate signal paths, then create rule-compliant wires and vias. Output: placed and routed geometry with a realizable clock and power network. L47-L48; L49-L52

6. Close and hand off GDSII

Extract interconnect parasitics; close setup/hold timing across required modes/corners, physical rules and power/reliability checks. DRC checks geometry, LVS checks extracted connectivity against the reference, and ERC checks electrical rules. ECOs trigger relevant rechecks. GDSII exports layout geometry; masks, fabrication, packaging and manufactured-device test follow the design handoff. L53-L54

Iteration and course approach

Why optimization appears more than once

Logic minimization changes functions' representations. Technology mapping changes the available gates and their costs. Placement and routing then expose wire resistance, capacitance and congestion. A smaller Boolean expression can map to slower cells or worse wiring; each stage updates the information used to judge the implementation.

Where it occurs	What changes	Why repeat it?
Before mapping	Boolean covers, factoring, shared logic and state encodings	Reduce structural cost while preserving specified behavior.
During/after mapping	Library cell choices, drive strengths and buffering	Meet modeled delay, transition, load and area requirements.
After placement, CTS and routing	Locations, buffers, cell sizes and selected connections	Repair violations using progressively more accurate physical information.

Resizing a gate changes both its delay and the load seen by its driver. A hold-fix buffer adds delay and can consume setup margin. Repair one violation, then recheck interacting paths; estimated placement parasitics and extracted routed parasitics provide different evidence. Timing-repair rationale

Models and checks that accompany the flow

Constraints: define clocks, external interface timing and justified timing exceptions; timing analysis needs the intended operating requirements. **Libraries:** supply cell functions, delay/check models and operating conditions. **Parasitics:** model interconnect loading and delay, with fidelity appropriate to the implementation stage. L26; L29; L32

Functional checks establish behavior under a specification and assumptions. **STA** checks modeled timing requirements. **DRC/LVS/ERC** check physical or electrical implementation rules. Passing one check does not establish the others. L08; L25; L28

How NPTEL builds the theory

Weeks	Main emphasis
1-2	IC technology, abstractions and the complete design flow
3	Verilog modeling and functional simulation
4-5	RTL synthesis, logic/FSM optimization and formal engines
6	Formal checking and technology libraries
7-8	STA, constraints, technology mapping and timing optimization
9-10	Power, DFT/ATPG/BIST and physical-design foundations
11-12	Floorplanning, placement, CTS, routing and closure

The teaching order introduces models before detailed optimization. Implementation uses those models together and revisits earlier decisions as physical information becomes available. These concepts carry into a Synopsys-based flow; commands and stage boundaries depend on the chosen tool environment.

Sources and deeper reading

NPTEL course outline gives the week sequence. Lesson links open the detailed notes. Timing repair explains interacting setup/hold checks; Full forms expands abbreviations. All 54 lessons through Week 12 are complete.